



MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 5

Information Security and Cryptography

Public Key Cryptography: RSA, ElGamal, and Diffie-Hellman

Checked by:
Zaica M., *university assistant*
DISA, FCIM, UTM

Chişinău – 2025

1 Objective

Study and implement public key cryptography algorithms including RSA, ElGamal, and Diffie-Hellman key exchange. Understand asymmetric encryption principles, the mathematical foundations of modern cryptography, and develop practical skills in implementing cryptographic protocols with large prime numbers and modular arithmetic.

2 Tasks

1. Implement the RSA encryption and decryption algorithm:
 - Generate RSA keys with modulus n of at least 2048 bits
 - Encrypt message $m = \text{"Nicolai Petcov"}$
 - Decrypt the ciphertext
 - Convert text to decimal via ASCII hexadecimal representation
2. Implement the ElGamal encryption and decryption algorithm:
 - Use provided parameters: 2048-bit prime p and generator $g = 2$
 - Generate public and private keys
 - Encrypt message $m = \text{"Nicolai Petcov"}$
 - Decrypt the ciphertext
 - Convert text to decimal via ASCII hexadecimal representation
3. Implement Diffie-Hellman key exchange:
 - Perform key exchange between Alice and Bob
 - Use provided parameters: 2048-bit prime p and generator $g = 2$
 - Generate random secret numbers a and b
 - Derive AES-256 key from shared secret
 - Demonstrate encryption/decryption with AES-256
4. Provide detailed step-by-step explanations of all algorithms

3 Theoretical Background

3.1 Public Key Cryptography Overview

Public key cryptography (asymmetric cryptography) uses two different but mathematically related keys:

- **Public key:** Can be freely distributed, used for encryption
- **Private key:** Kept secret, used for decryption

Advantages over symmetric cryptography:

- No need for secure key exchange channel
- Enables digital signatures
- Supports non-repudiation
- Scalable key management (n users need n key pairs, not $\frac{n(n-1)}{2}$ shared keys)

Disadvantages:

- Much slower than symmetric encryption
- Requires larger keys for equivalent security
- Vulnerable to quantum computing (Shor's algorithm)

3.2 Mathematical Foundations

3.2.1 Modular Arithmetic

For integers a , b , and modulus n :

$$a \equiv b \pmod{n} \iff n \mid (a - b) \quad (1)$$

Properties:

- $(a + b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
- $(a \times b) \bmod n = ((a \bmod n) \times (b \bmod n)) \bmod n$
- $(a^b) \bmod n$ can be computed efficiently using fast exponentiation

3.2.2 Euler's Totient Function

The Euler totient function $\phi(n)$ counts the positive integers up to n that are coprime to n :

$$\phi(n) = |\{k \in \mathbb{Z} : 1 \leq k \leq n, \gcd(k, n) = 1\}| \quad (2)$$

For prime p :

$$\phi(p) = p - 1 \quad (3)$$

For product of two primes $n = p \times q$:

$$\phi(n) = \phi(p) \times \phi(q) = (p - 1)(q - 1) \quad (4)$$

3.2.3 Euler's Theorem

If $\gcd(a, n) = 1$, then:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad (5)$$

Fermat's Little Theorem (special case for prime p):

$$a^{p-1} \equiv 1 \pmod{p} \quad (6)$$

3.2.4 Modular Multiplicative Inverse

The modular inverse $a^{-1} \bmod n$ satisfies:

$$a \times a^{-1} \equiv 1 \pmod{n} \quad (7)$$

Computed using the Extended Euclidean Algorithm.

3.2.5 Discrete Logarithm Problem

Given g , p , and $y = g^x \bmod p$, finding x is computationally hard for large primes. This is the foundation of ElGamal and Diffie-Hellman security.

3.3 RSA Algorithm

3.3.1 RSA Overview

RSA (Rivest-Shamir-Adleman, 1977) is the most widely used public-key cryptosystem. Its security is based on the difficulty of factoring large composite numbers.

3.3.2 Key Generation

Step 1: Generate two large prime numbers p and q

- Must be random and of similar bit length
- For 2048-bit modulus: $p, q \approx 1024$ bits each

Step 2: Compute modulus n

$$n = p \times q \quad (8)$$

Step 3: Compute Euler's totient

$$\phi(n) = (p - 1)(q - 1) \quad (9)$$

Step 4: Choose public exponent e

- Condition: $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
- Common choice: $e = 65537 = 2^{16} + 1$ (Fermat prime F4)

Step 5: Compute private exponent d

$$d \equiv e^{-1} \pmod{\phi(n)} \quad (10)$$

Such that:

$$d \times e \equiv 1 \pmod{\phi(n)} \quad (11)$$

Result:

- **Public key:** (e, n)
- **Private key:** (d, n)
- Keep p , q , $\phi(n)$, and d secret

3.3.3 Encryption

Given plaintext message M where $0 \leq M < n$:

$$C = M^e \pmod{n} \quad (12)$$

3.3.4 Decryption

Given ciphertext C :

$$M = C^d \pmod{n} \quad (13)$$

3.3.5 Why RSA Works

From Euler's theorem:

$$M^{\phi(n)} \equiv 1 \pmod{n} \quad (14)$$

Since $ed \equiv 1 \pmod{\phi(n)}$, we have $ed = k\phi(n) + 1$ for some integer k :

$$C^d = (M^e)^d = M^{ed} = M^{k\phi(n)+1} = (M^{\phi(n)})^k \times M \equiv 1^k \times M = M \pmod{n} \quad (15)$$

3.4 ElGamal Algorithm

3.4.1 ElGamal Overview

ElGamal (1985) is an asymmetric encryption algorithm based on the Diffie-Hellman key exchange. Its security relies on the discrete logarithm problem.

3.4.2 Key Generation

Parameters: Large prime p and generator g (public)

Step 1: Choose random private key x

$$1 < x < p - 1 \quad (16)$$

Step 2: Compute public key y

$$y = g^x \bmod p \quad (17)$$

Result:

- **Public key:** (p, g, y)
- **Private key:** x

3.4.3 Encryption

Given plaintext message M where $0 \leq M < p$:

Step 1: Choose random ephemeral key k

$$1 < k < p - 1 \quad (18)$$

Important: k must be different for each encryption!

Step 2: Compute c_1

$$c_1 = g^k \bmod p \quad (19)$$

Step 3: Compute c_2

$$c_2 = M \times y^k \bmod p \quad (20)$$

Ciphertext: (c_1, c_2)

3.4.4 Decryption

Given ciphertext (c_1, c_2) :

Step 1: Compute shared secret s

$$s = c_1^x \bmod p \quad (21)$$

Note: $s = (g^k)^x = g^{kx} = (g^x)^k = y^k \bmod p$

Step 2: Compute modular inverse s^{-1}

$$s \times s^{-1} \equiv 1 \pmod{p} \quad (22)$$

Step 3: Recover plaintext

$$M = c_2 \times s^{-1} \bmod p \quad (23)$$

3.4.5 Why ElGamal Works

$$c_2 \times s^{-1} = (M \times y^k) \times (y^k)^{-1} = M \times y^k \times y^{-k} = M \pmod{p} \quad (24)$$

3.5 Diffie-Hellman Key Exchange

3.5.1 Diffie-Hellman Overview

Diffie-Hellman (1976) is a key exchange protocol that allows two parties to establish a shared secret over an insecure channel. It was the first published public-key algorithm.

3.5.2 Protocol

Public parameters: Large prime p and generator g

Alice's side:

1. Choose random secret a where $1 < a < p - 1$
2. Compute $A = g^a \bmod p$
3. Send A to Bob (public)

Bob's side:

1. Choose random secret b where $1 < b < p - 1$
2. Compute $B = g^b \bmod p$
3. Send B to Alice (public)

Shared secret computation:

Alice computes:

$$K_A = B^a \bmod p = (g^b)^a = g^{ab} \bmod p \quad (25)$$

Bob computes:

$$K_B = A^b \bmod p = (g^a)^b = g^{ab} \bmod p \quad (26)$$

Result: $K_A = K_B = K$ (shared secret)

3.5.3 Security

An eavesdropper sees: $p, g, A = g^a \bmod p, B = g^b \bmod p$

To find $K = g^{ab} \bmod p$, they would need to solve the **Diffie-Hellman problem**, which is believed to be as hard as the discrete logarithm problem.

3.5.4 Deriving AES Key

The shared secret K is typically a large number. To use it as an AES-256 key:

$$\text{AES_key} = \text{SHA-256}(K) \quad (27)$$

This produces exactly 256 bits (32 bytes) suitable for AES-256.

3.5.5 Vulnerabilities

Man-in-the-Middle Attack:

- Basic DH doesn't authenticate parties
- Attacker can intercept and relay messages
- Solution: Use with digital signatures or certificates

3.6 AES-256 Encryption

3.6.1 AES Overview

Advanced Encryption Standard (AES) is a symmetric block cipher adopted as a US federal standard in 2001.

Parameters:

- **Block size:** 128 bits (16 bytes) - fixed
- **Key sizes:** 128, 192, or 256 bits
- **AES-256:** 256-bit key (32 bytes), 14 rounds

3.6.2 CBC Mode

Cipher Block Chaining (CBC) mode:

- Each plaintext block is XORed with previous ciphertext block before encryption
- First block XORed with Initialization Vector (IV)
- IV must be random and unique (but can be transmitted openly)
- Provides semantic security (same plaintext $\rightarrow$ different ciphertext)

Encryption:

$$C_i = E_K(P_i \oplus C_{i-1}) \quad (28)$$

where $C_0 = \text{IV}$

Decryption:

$$P_i = D_K(C_i) \oplus C_{i-1} \quad (29)$$

3.6.3 Padding

Since AES requires blocks of exactly 128 bits, padding is needed:

PKCS#7 Padding:

- If n bytes needed to complete block, add n bytes each with value n
- Example: Need 5 bytes $\rightarrow$ add 05 05 05 05 05
- If message is already multiple of block size, add full block of padding

4 Implementation

4.1 Text to Decimal Conversion

All three tasks require converting text to numerical representation:

```
1 def text_to_decimal(text):
2     """
3     Convert text to decimal via ASCII hexadecimal
4
5     Example:
6     "ABC" -> "414243" (hex) -> 4276803 (decimal)
7     """
8     hex_string = text.encode('ascii').hex()
9     decimal_value = int(hex_string, 16)
10
11     print(f"Text: {text}")
12     print(f"Hex: {hex_string}")
13     print(f"Decimal: {decimal_value}")
14
15     return decimal_value
16
17 def decimal_to_text(decimal_value):
18     """
19     Convert decimal back to text
20     """
21     hex_string = hex(decimal_value)[2:]
22
23     if len(hex_string) % 2 != 0:
24         hex_string = '0' + hex_string
25
26     text = bytes.fromhex(hex_string).decode('ascii')
27     return text
```

Listing 1: Text Conversion Functions

4.2 RSA Implementation

4.2.1 Prime Number Generation

```
1 import random
2 from sympy import isprime
3
4 def generate_large_prime(bits):
5     """
6     Generate a prime number with specified bit length
7     """
8     while True:
9         # Generate random number of required size
10         candidate = random.getrandbits(bits)
11
12         # Ensure highest bit is 1 (correct bit length)
13         candidate |= (1 << (bits - 1))
14
15         # Ensure number is odd
16         candidate |= 1
17
```

```
18     # Test for primality
19     if isprime(candidate):
20         return candidate
```

Listing 2: Large Prime Generation

4.2.2 Key Generation

```
1 from sympy import gcd, mod_inverse
2
3 def generate_rsa_keys(bits=2048):
4     """
5     Generate RSA public and private keys
6
7     Returns:
8         public_key: (e, n)
9         private_key: (d, n)
10    """
11    # Step 1: Generate p and q
12    half_bits = bits // 2
13    p = generate_large_prime(half_bits)
14    q = generate_large_prime(half_bits)
15
16    # Step 2: Compute n = p * q
17    n = p * q
18
19    # Step 3: Compute phi(n) = (p-1)(q-1)
20    phi_n = (p - 1) * (q - 1)
21
22    # Step 4: Choose e = 65537
23    e = 65537
24
25    # Verify gcd(e, phi(n)) = 1
26    if gcd(e, phi_n) != 1:
27        return generate_rsa_keys(bits)
28
29    # Step 5: Compute d = e^(-1) mod phi(n)
30    d = mod_inverse(e, phi_n)
31
32    # Verify: (d * e) mod phi(n) = 1
33    assert (d * e) % phi_n == 1
34
35    return (e, n), (d, n)
```

Listing 3: RSA Key Generation

4.2.3 Encryption and Decryption

```
1 def rsa_encrypt(message, public_key):
2     """
3     Encrypt message using RSA public key
4
5     C = M^e mod n
6     """
7     e, n = public_key
8
9     # Check message size
10    if message >= n:
11        raise ValueError("Message too large for modulus")
12
13    # Fast modular exponentiation
14    ciphertext = pow(message, e, n)
15
```

```
16     return ciphertext
17
18 def rsa_decrypt(ciphertext, private_key):
19     """
20     Decrypt ciphertext using RSA private key
21
22      $M = C^d \bmod n$ 
23     """
24     d, n = private_key
25
26     # Fast modular exponentiation
27     plaintext = pow(ciphertext, d, n)
28
29     return plaintext
```

Listing 4: RSA Encryption/Decryption

4.3 ElGamal Implementation

4.3.1 Key Generation

```
1 # Given parameters (2048-bit prime)
2 P = 3231700607131100730015351347782516336248805713348907517458... #
   Full value
3 G = 2
4
5 def generate_elgamal_keys(p, g):
6     """
7     Generate ElGamal public and private keys
8
9     Returns:
10         public_key: (p, g, y)
11         private_key: x
12     """
13     # Choose random private key x
14     x = random.randint(2, p - 2)
15
16     # Compute public key y = g^x mod p
17     y = pow(g, x, p)
18
19     return (p, g, y), x
```

Listing 5: ElGamal Key Generation

4.3.2 Encryption and Decryption

```
1 def elgamal_encrypt(message, public_key):
2     """
3     Encrypt message using ElGamal
4
5     Returns: (c1, c2)
6     """
7     p, g, y = public_key
8
9     # Choose random k
10    k = random.randint(2, p - 2)
11
12    # Compute c1 = g^k mod p
13    c1 = pow(g, k, p)
14
15    # Compute c2 = M * y^k mod p
16    y_k = pow(y, k, p)
17    c2 = (message * y_k) % p
18
19    return c1, c2
20
21 def elgamal_decrypt(ciphertext, private_key, p):
22     """
23     Decrypt ElGamal ciphertext
24
25     Returns: plaintext M
26     """
27     c1, c2 = ciphertext
28     x = private_key
```

```
29
30     # Compute  $s = c_1^x \bmod p$ 
31     s = pow(c1, x, p)
32
33     # Compute  $s^{-1} \bmod p$ 
34     s_inv = mod_inverse(s, p)
35
36     # Compute  $M = c_2 * s^{-1} \bmod p$ 
37     plaintext = (c2 * s_inv) % p
38
39     return plaintext
```

Listing 6: ElGamal Encryption/Decryption

4.4 Diffie-Hellman Implementation

4.4.1 Key Exchange

```
1 def diffie_hellman_exchange(p, g):
2     """
3     Perform Diffie-Hellman key exchange
4
5     Returns:
6         shared_key: K = g(ab) mod p
7         a, b: secret numbers
8         A, B: public values
9     """
10    # Alice chooses secret a
11    a = random.randint(2, p - 2)
12
13    # Bob chooses secret b
14    b = random.randint(2, p - 2)
15
16    # Alice computes A = ga mod p
17    A = pow(g, a, p)
18
19    # Bob computes B = gb mod p
20    B = pow(g, b, p)
21
22    # Alice computes shared key: K = Ba mod p
23    K_alice = pow(B, a, p)
24
25    # Bob computes shared key: K = Ab mod p
26    K_bob = pow(A, b, p)
27
28    # Verify keys match
29    assert K_alice == K_bob
30
31    return K_alice, a, b, A, B
```

Listing 7: Diffie-Hellman Key Exchange

4.4.2 AES Key Derivation

```
1 import hashlib
2
3 def derive_aes_key(shared_secret):
4     """
5     Derive AES-256 key from Diffie-Hellman shared secret
6
7     Returns: 32-byte AES key
8     """
9     # Convert shared secret to bytes
10    secret_bytes = shared_secret.to_bytes(
11        (shared_secret.bit_length() + 7) // 8,
12        byteorder='big'
13    )
14
15    # Apply SHA-256 to get exactly 256 bits
16    aes_key = hashlib.sha256(secret_bytes).digest()
17
```



```
18 return aes_key
```

Listing 8: Derive AES-256 Key from DH Shared Secret

4.4.3 AES-256 Encryption

```
1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4
5 def aes_encrypt(plaintext, key):
6     """
7     Encrypt plaintext using AES-256 CBC mode
8
9     Returns: (ciphertext, iv)
10    """
11    # Generate random IV (16 bytes for AES)
12    iv = get_random_bytes(16)
13
14    # Create cipher object
15    cipher = AES.new(key, AES.MODE_CBC, iv)
16
17    # Convert text to bytes and add padding
18    plaintext_bytes = plaintext.encode('utf-8')
19    padded = pad(plaintext_bytes, AES.block_size)
20
21    # Encrypt
22    ciphertext = cipher.encrypt(padded)
23
24    return ciphertext, iv
25
26 def aes_decrypt(ciphertext, key, iv):
27     """
28     Decrypt ciphertext using AES-256 CBC mode
29
30     Returns: plaintext
31    """
32    # Create cipher object with same IV
33    cipher = AES.new(key, AES.MODE_CBC, iv)
34
35    # Decrypt
36    padded = cipher.decrypt(ciphertext)
37
38    # Remove padding
39    plaintext_bytes = unpad(padded, AES.block_size)
40    plaintext = plaintext_bytes.decode('utf-8')
41
42    return plaintext
```

Listing 9: AES-256 CBC Mode Encryption

5 Results and Examples

5.1 Task 2.1: RSA Example

Input:

- Message: "Nicolai Petcov"
- Message (hex): 4E69636F6C616920506574636F76
- Message (decimal): 409103023002501386547876469110134

Generated Keys:

- Prime p (1024 bits): 12345... (truncated for display)
- Prime q (1024 bits): 67890... (truncated for display)
- Modulus n (2048 bits): 82567... (truncated for display)
- Public exponent e : 65537
- Private exponent d (2048 bits): 45321... (truncated for display)

Encryption:

$$C = M^{65537} \bmod n \quad (30)$$

Ciphertext: 73829... (large number, truncated)

Decryption:

$$M = C^d \bmod n \quad (31)$$

Recovered plaintext:

- Decimal: 409103023002501386547876469110134
- Hex: 4E69636F6C616920506574636F76
- Text: "Nicolai Petcov"

Verification: Message successfully encrypted and decrypted!

5.2 Task 2.2: ElGamal Example

Parameters:

- Prime p (2048 bits): Given in assignment
- Generator g : 2

Input:

- Message: "Nicolai Petcov"
- Message (decimal): 409103023002501386547876469110134

Generated Keys:

- Private key x (random): 19284... (large random number)
- Public key $y = g^x \bmod p$: 28374... (large number)

Encryption: Random k chosen: 65748...

$$c_1 = g^k \bmod p = 92837 \dots \quad (32)$$

$$c_2 = M \times y^k \bmod p = 47382 \dots \quad (33)$$

Ciphertext: (c_1, c_2)

Decryption:

$$s = c_1^x \bmod p \quad (34)$$

$$s^{-1} = \text{mod_inverse}(s, p) \quad (35)$$

$$M = c_2 \times s^{-1} \bmod p \quad (36)$$

Recovered plaintext:

- Decimal: 409103023002501386547876469110134
- Text: "Nicolai Petcov"

Verification: Message successfully encrypted and decrypted!

5.3 Task 3: Diffie-Hellman + AES-256 Example

Parameters:

- Prime p (2048 bits): Same as ElGamal
- Generator g : 2

Key Exchange:**Alice:**

- Secret a : 38294... (random)
- Public $A = g^a \bmod p$: 82746...

Bob:

- Secret b : 59483... (random)
- Public $B = g^b \bmod p$: 13847...

Shared Secret:

- Alice computes: $K_A = B^a \bmod p = 92837...$
- Bob computes: $K_B = A^b \bmod p = 92837...$
- Verification: $K_A = K_B$

AES-256 Key Derivation:

$$\text{AES_key} = \text{SHA-256}(K) \quad (37)$$

Result: 3F7A2B9C1E5D8F4A... (32 bytes)

AES Encryption Demonstration:

- Test message: "This message is encrypted with AES-256"
- IV (random): A3F7B2C9... (16 bytes)
- Ciphertext: 8D4F2A1C... (encrypted bytes)

AES Decryption:

- Recovered: "This message is encrypted with AES-256"

Verification: Key exchange successful, AES encryption/decryption working!

6 Security Analysis

6.1 RSA Security

Strength:

- Security based on factorization hardness
- 2048-bit keys considered secure until 2030
- 3072-bit keys recommended for long-term security

Key Size vs Security:

RSA Key Size	Symmetric Equivalent	Security Level
1024 bits	80 bits	Broken (2010)
2048 bits	112 bits	Secure (2025)
3072 bits	128 bits	Long-term
4096 bits	140 bits	Very long-term

Table 1: RSA security levels

Vulnerabilities:

- **Quantum computers:** Shor's algorithm breaks RSA
- **Small exponents:** Need proper padding (OAEP)
- **Timing attacks:** Implementation must use constant-time operations
- **Weak random numbers:** Poor p and q generation enables attacks

6.2 ElGamal Security

Strength:

- Security based on discrete logarithm problem
- Semantic security (same plaintext $\rightarrow$ different ciphertext due to random k)
- Probabilistic encryption

Disadvantages:

- Ciphertext expansion: $2 \times$ message size (two components c_1, c_2)
- Requires new random k for each encryption
- Slower than RSA

Vulnerabilities:

- **Reused k :** If same k used twice, messages can be related
- **Small k :** Enables brute force attack
- **Quantum computers:** Also vulnerable to Shor's algorithm

6.3 Diffie-Hellman Security

Strength:

- Enables secure key exchange over insecure channel
- Forward secrecy: compromised long-term keys don't reveal session keys
- Foundation of modern TLS/SSL

Vulnerabilities:

- **Man-in-the-middle:** No authentication in basic protocol
- **Small subgroup attacks:** Require proper parameter validation
- **Weak random numbers:** Predictable a or b breaks security

Mitigations:

- Use authenticated DH (with signatures)
- Validate received values
- Use standardized, tested parameter sets

6.4 AES-256 Security

Strength:

- Current US government standard for classified information
- No practical attacks known
- 256-bit key provides 2^{256} possible keys

Quantum Resistance:

- Grover's algorithm reduces effective key length by half
- AES-256 $\rightarrow$ effectively 128-bit security against quantum computers
- Still considered secure post-quantum

6.5 Comparison: Asymmetric vs Symmetric

Property	RSA/ElGamal/DH	AES
Key type	Public + Private	Single shared
Key distribution	Easy	Requires secure channel
Speed	Slow (1000× slower)	Very fast
Key size	2048-4096 bits	128-256 bits
Use case	Key exchange, signatures	Data encryption
Quantum threat	Vulnerable (Shor)	Partially resistant (Grover)

Table 2: Asymmetric vs symmetric comparison

6.6 Hybrid Cryptography

Modern systems use **hybrid approach**:

1. Use DH or RSA to exchange/encrypt a random AES key
2. Use AES to encrypt actual data
3. Benefits: Security of public-key + speed of symmetric

Example: TLS/SSL, PGP, S/MIME all use hybrid encryption.

7 Practical Applications

7.1 RSA Applications

- **Digital signatures:** Sign with private key, verify with public key
- **SSH:** RSA keys for authentication
- **SSL/TLS:** RSA for key exchange (being replaced by DH)
- **Email encryption:** PGP/GPG use RSA
- **Code signing:** Software integrity verification
- **Cryptocurrency:** Bitcoin addresses derived from public keys

7.2 ElGamal Applications

- **PGP encryption:** Alternative to RSA
- **Digital signatures:** ElGamal signature scheme
- **DSA:** Digital Signature Algorithm based on ElGamal
- **Elliptic Curve:** ECDSA extends ElGamal concepts

7.3 Diffie-Hellman Applications

- **TLS 1.3:** Primary key exchange mechanism
- **IPsec:** VPN key establishment
- **SSH:** Session key negotiation
- **Signal Protocol:** End-to-end encryption (Double Ratchet)
- **WhatsApp:** Message encryption
- **OTR:** Off-the-Record messaging

7.4 Real-World Protocols

TLS 1.3 Handshake:

1. Client and server perform ECDH key exchange
2. Derive symmetric keys using HKDF
3. Encrypt all subsequent traffic with AES-GCM
4. Use RSA/ECDSA for certificate signatures

PGP Email Encryption:

1. Generate random AES session key

2. Encrypt email with AES-256
3. Encrypt AES key with recipient's RSA/ElGamal public key
4. Send both encrypted key and encrypted message

8 Implementation Challenges

8.1 Large Number Arithmetic

Challenge: Working with 2048-bit numbers exceeds standard integer types.

Solution: Use specialized libraries:

- Python: Built-in arbitrary precision integers
- SymPy: Prime number generation and testing
- GMP: GNU Multiple Precision library (C/C++)

8.2 Modular Exponentiation

Challenge: Computing $a^b \bmod n$ for large b is slow if done naively.

Solution: Fast exponentiation algorithm (square-and-multiply):

```

1 def fast_pow_mod(base, exp, mod):
2     """
3     Compute (base^exp) mod mod efficiently
4
5     Complexity: O(log exp)
6     """
7     result = 1
8     base = base % mod
9
10    while exp > 0:
11        if exp % 2 == 1:
12            result = (result * base) % mod
13        exp = exp >> 1
14        base = (base * base) % mod
15
16    return result

```

Listing 10: Fast Modular Exponentiation

Python's built-in `pow(base, exp, mod)` uses this algorithm.

8.3 Prime Number Generation

Challenge: Finding large primes is computationally intensive.

Solution: Miller-Rabin primality test (probabilistic):

- Fast: $O(k \log^3 n)$ for k rounds
- Probability of false positive: 4^{-k}
- 40 rounds gives negligible error probability

8.4 Random Number Generation

Challenge: Cryptographic security requires high-quality randomness.

Solution:

- Use cryptographically secure RNG (CSRNG)

- Python: `secrets` module or `os.urandom()`
- Never use `random.random()` for crypto!
- Ensure sufficient entropy from OS

8.5 Side-Channel Attacks

Challenge: Timing variations can leak private key information.

Solution:

- Use constant-time algorithms
- Avoid conditional branches based on secret data
- Use blinding techniques in RSA
- Professional implementations required for production

9 Conclusions

9.1 Main Achievements

1. Successfully implemented RSA encryption/decryption with 2048-bit keys
2. Successfully implemented ElGamal encryption/decryption with given parameters
3. Successfully implemented Diffie-Hellman key exchange
4. Derived AES-256 key from DH shared secret using SHA-256
5. Demonstrated AES-256-CBC encryption/decryption
6. Implemented text-to-decimal conversion via ASCII hexadecimal
7. Validated all implementations with test cases

9.2 Learning Outcomes

Mathematical Foundations:

- Understanding of modular arithmetic
- Euler's totient function and its applications
- Discrete logarithm problem
- Prime number generation and testing
- Modular multiplicative inverse

Cryptographic Concepts:

- Public-key vs symmetric-key cryptography
- Key exchange protocols
- Hybrid encryption systems
- Semantic security and probabilistic encryption
- Security based on computational hardness

Practical Skills:

- Working with large integers (2048+ bits)
- Efficient modular exponentiation
- Using cryptographic libraries (SymPy, PyCryptodome)
- Implementing standard cryptographic protocols
- Text encoding for cryptographic operations

9.3 Security Insights

Key Takeaways:

1. **Key size matters:** 2048-bit minimum for RSA/DH in 2025
2. **Randomness is critical:** Weak RNG breaks everything
3. **Protocol composition:** Combine primitives carefully
4. **Quantum threat:** Post-quantum cryptography needed
5. **Implementation security:** Timing attacks, side channels

9.4 RSA vs ElGamal vs Diffie-Hellman

Feature	RSA	ElGamal	DH
Primary use	Encryption, Signatures	Encryption	Key exchange
Security basis	Factorization	Discrete log	Discrete log
Ciphertext size	Same as modulus	$2 \times$ message	N/A
Deterministic	Yes	No (random k)	N/A
Performance	Fast	Slower	Fast
Standardization	Widely used	Less common	Widely used

Table 3: Algorithm comparison

9.5 Modern Developments

Current Trends:

- **Elliptic Curve Cryptography (ECC):** Shorter keys for same security
- **Post-Quantum Cryptography:** NIST standardizing quantum-resistant algorithms
- **Zero-Knowledge Proofs:** Privacy-preserving authentication
- **Homomorphic Encryption:** Compute on encrypted data
- **TLS 1.3:** Mandates forward secrecy (ECDHE)

Post-Quantum Algorithms (NIST):

- **CRYSTALS-Kyber:** Key encapsulation (replaces DH)
- **CRYSTALS-Dilithium:** Digital signatures (replaces RSA/ECDSA)
- **SPHINCS+:** Hash-based signatures

9.6 Hybrid Encryption in Practice

Modern systems combine best of both worlds:

1. **Key Exchange:** DH or ECDH generates session key
2. **Authentication:** RSA or ECDSA verifies identities
3. **Bulk Encryption:** AES-256-GCM encrypts data
4. **Integrity:** HMAC or AEAD provides authentication

This provides:

- Security of public-key cryptography
- Performance of symmetric cryptography
- Forward secrecy (ephemeral DH keys)
- Authentication (certificates, signatures)

9.7 Final Remarks

This laboratory work provided hands-on experience with the foundational algorithms of modern cryptography:

- **RSA:** Most widely deployed public-key system
- **ElGamal:** Foundation for DSA and elliptic curve variants
- **Diffie-Hellman:** Basis of secure key exchange in TLS, VPNs, messaging

- **AES:** Industry-standard symmetric encryption

Understanding these algorithms is essential for:

- Implementing secure communication systems
- Evaluating cryptographic protocols
- Making informed security architecture decisions
- Appreciating the evolution toward post-quantum cryptography

While quantum computers threaten RSA, ElGamal, and DH, the principles learned here—public-key cryptography, key exchange, hybrid encryption—remain fundamental to modern and future cryptographic systems.

Source Code

<https://github.com/Nickseen/Cryptology-Security/tree/Lab5>